



Desenvolvimento Web

Profª Sabrina B. Moreira



JavaScript

Aula 20.1: orientação a eventos de tags html

Eventos

JS: eventos

- ❖ Os eventos representam ações que acontecem na página, como cliques, digitação, envio de formulários ou movimentação do mouse.
- ❖ O JavaScript pode "escutar" esses eventos e executar funções específicas quando eles ocorrerem, tornando a aplicação interativa.

```
<body>
  <p onclick="onClick()"> Click here! </p>
  <script>
    function onClick ()
    {
      alert ( "Welcome to Sitechecker!" );
    }
  </script>
</body>
```

HTML onclick Attribute

JS: eventos

- ❖ Existem vários eventos que podem ocorrer dentro de uma página **HTML**, como clicar com o mouse, digitar informações ou até sair de uma página. Quando carregamos uma página, o evento *OnLoad* será acionado.
- ❖ Os eventos são realizados em **tags HTML** e por isso são atrelados a elas, **dependendo da ação do usuário** na página, o evento pode ou não acontecer.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Wikitechy HTML onload attribute</title>
    <script>
      1 ← function onLoad() {
          alert("Wikitechy Page is loaded")
        }
      </script> 2
    </head>
    <body onload="onLoad()">
      <h2>Welcome to Wikitechy!</h2>
    </body>
  </html>
```

Fonte: wikitechy, 2026

JS: object event

- ❖ Sempre que um evento ocorre em uma página web, o navegador cria automaticamente um objeto chamado *event*, que contém diversas informações sobre a ação realizada pelo usuário.
- ❖ Esse objeto permite identificar, por exemplo, qual elemento disparou o evento, qual **tecla foi pressionada** ou o tipo de evento ocorrido, possibilitando a criação de comportamentos dinâmicos e personalizados na aplicação.

```
pressure: 0
pseudoTarget: null
relatedTarget: null
returnValue: true
screenX: 89
screenY: 564
shiftKey: false
▶ sourceCapabilities: InputDeviceCapa
▶ srcElement: button#btnMouse
tangentialPressure: 0
▶ target: button#btnMouse
tiltX: 0
tiltY: 0
timeStamp: 3063.199999988079
toElement: null
twist: 0
type: "click"
▶ view: Window {window: Window, self:
  which: 1
  width: 1
  x: 85
  y: 484
▶ [[Prototype]]: PointerEvent
```

JS: object event

- ❖ As propriedades disponíveis no objeto *event* variam conforme o tipo de evento (mouse, teclado, formulário...

Nome	Descrição	Exemplo
event.type	Retorna o tipo do evento ocorrido.	"click", "keydown"
event.target	Retorna o elemento HTML que disparou o evento.	<button id="btnMouse">
event.currentTarget	Retorna o elemento ao qual o evento foi associado.	<button id="btnMouse">
event.preventDefault()	Impede o comportamento padrão do navegador.	Cancelar envio de formulário
event.stopPropagation()	Impede que o evento continue se propagando para elementos pais.	Interromper propagação de clique

JS: object event

Nome	Descrição	Exemplo
event.key	Retorna a tecla pressionada pelo usuário.	"a", "Enter"
event.code	Retorna o código físico da tecla pressionada.	"KeyA", "Enter"
event.clientX	Retorna a posição horizontal do mouse na janela.	250
event.clientY	Retorna a posição vertical do mouse na janela.	180
event.altKey	Indica se a tecla Alt estava pressionada.	true / false
event.ctrlKey	Indica se a tecla Ctrl estava pressionada.	true / false
event.shiftKey	Indica se a tecla Shift estava pressionada.	true / false

Eventos de mouse

JS: eventos de mouse

- ❖ Os eventos de mouse permitem **detectar interações** realizadas com o cursor sobre os elementos da página.
- ❖ Entre os principais eventos estão *onclick*, acionado quando o usuário **clica** em um elemento.
- ❖ O *onmouseover*, executado quando o **cursor** passa sobre um elemento e *onmouseout*, disparado quando o **cursor** sai da área de um elemento.

```
currentTarget: null
defaultPrevented: false
detail: 0
eventPhase: 0
▶ fromElement: body
layerX: 218
layerY: 513
metaKey: false
movementX: 0
movementY: 0
offsetX: 209
offsetY: -2
pageX: 218
pageY: 513
pseudoTarget: null
▶ relatedTarget: body
returnValue: true
screenX: 222
screenY: 593
shiftKey: false
▶ sourceCapabilities: InputDeviceCapabilities
▶ srcElement: div#areaMouse
▶ target: div#areaMouse
timestamp: 1583.1000000238419
▶ toElement: div#areaMouse
type: "mouseover"
```

JS: Principais Eventos HTML - mouse

Nome	Descrição	Gatilho
onclick	Ocorre quando o usuário clica em um elemento	Clique em um botão
ondblclick	Ocorre quando o usuário dá um duplo clique	Duplo clique em um elemento
onmousedown	Ocorre quando um botão do mouse é pressionado	Pressionar o mouse
onmouseup	Ocorre quando um botão do mouse é liberado	Soltar o mouse
onmouseover	Ocorre quando o ponteiro entra sobre um elemento	Passar o mouse sobre uma imagem
onmouseout	Ocorre quando o ponteiro sai de um elemento	Retirar o mouse de um botão
onmousemove	Ocorre enquanto o mouse se move sobre um elemento	Rastrear posição do cursor

Eventos de teclado

JS: eventos de teclado

- ❖ Os eventos de teclado são utilizados para capturar ações realizadas pelo usuário por meio das **teclas do teclado**, sendo muito comuns em formulários, atalhos e validações em tempo real.
- ❖ Os principais são *onkeydown*, executado quando uma tecla é pressionada. *onkeyup*, acionado quando a tecla é liberada e *onkeypress*, disparado quando uma tecla que **produz caracteres** é pressionada (embora atualmente seja considerado obsoleto).

```
currentTarget: null
defaultPrevented: false
detail: 0
eventPhase: 0
isComposing: false
key: "e"
keyCode: 69
location: 0
metaKey: false
pseudoTarget: null
repeat: false
returnValue: true
shiftKey: false
▶ sourceCapabilities: InputDevi
▶ srcElement: textarea#comentar
▶ target: textarea#comentario
timestamp: 21715.5
type: "keydown"
▶ view: Window {window: Window,
which: 69
▶ [[Prototype]]: KeyboardEvent
```

JS: Principais Eventos HTML - teclado

Nome	Descrição	Gatilho
onkeydown	Ocorre quando uma tecla é pressionada	Pressionar qualquer tecla
onkeyup	Ocorre quando uma tecla é liberada	Soltar uma tecla
onkeypress	Ocorre quando uma tecla que produz caractere é pressionada (obsoleto)	Digitar uma letra

Fonte: autora, 2026

Eventos de formulário

JS: eventos de formulário

- ❖ Permitem **monitorar e controlar** a interação do usuário com campos de entrada de dados. Eles são essenciais para **validação e processamento** das informações antes do envio.
- ❖ Os principais são *onsubmit*, executado quando o formulário é enviado. *onchange*, acionado quando o **valor de um campo é alterado** e *onfocus*, disparado quando um campo recebe o foco do usuário.

```
isTrusted: true
bubbles: false
cancelBubble: false
cancelable: false
composed: true
currentTarget: null
defaultPrevented: false
detail: 0
eventPhase: 0
pseudoTarget: null
relatedTarget: null
returnValue: true
sourceCapabilities: InputDevice
srcElement: input#nome
target: input#nome
timeStamp: 16883.600000023842
type: "focus"
view: Window {window: Window,
```

JS: Principais Eventos HTML - formulário

Nome	Descrição	Gatilho
onsubmit	Ocorre quando um formulário é enviado	Clicar no botão enviar
onchange	Ocorre quando o valor de um campo é alterado	Alterar um campo de texto
oninput	Ocorre sempre que o usuário digita em um campo	Digitação em tempo real
onfocus	Ocorre quando um elemento recebe foco	Clicar em um campo
onblur	Ocorre quando um elemento perde o foco	Sair de um campo

Fonte: autora, 2026

Eventos de janela

JS: eventos de janela

- ❖ Estão relacionados ao comportamento do navegador e da própria página web.
- ❖ Eles permitem executar ações em momentos específicos do ciclo de vida da página.
- ❖ Entre os principais destacam-se *onload*, executado quando toda a página é carregada, *onresize*, acionado quando a janela do navegador é redimensionada e *onscroll*, disparado quando o usuário realiza a rolagem da página.

```
ClipboardEvent {isTrusted: true, cl  
rentTarget: textarea#comentario, ...}  
  isTrusted: true  
  bubbles: true  
  cancelBubble: false  
  cancelable: true  
▶ clipboardData: DataTransfer {drop  
  composed: true  
  currentTarget: null  
  defaultPrevented: false  
  eventPhase: 0  
  returnValue: true  
▶ srcElement: textarea#comentario  
▶ target: textarea#comentario  
  timeStamp: 19512.3999999976158  
  type: "copy"
```

JS: Principais Eventos HTML - janela

Nome	Descrição	Gatilho
onload	Ocorre quando a página termina de carregar	Carregamento completo da página
onresize	Ocorre quando a janela é redimensionada	Alterar tamanho da janela
onscroll	Ocorre quando a página é rolada	Rolar a página

Fonte: autora, 2026

Eventos de conteúdo

JS: eventos de clipboard

- ❖ Os eventos de *clipboard* permitem **capturar ações** relacionadas à área de transferência do sistema operacional, como **copiar, recortar e colar** conteúdos.
- ❖ Os principais são *oncopy*, acionado ao **copiar** um conteúdo, *oncut* executado ao **recortar** um conteúdo e *onpaste* disparado quando o usuário **cola** informações em um campo ou elemento.

```
ClipboardEvent {isTrusted: true, cl  
rentTarget: textarea#comentario, ...}  
  isTrusted: true  
  bubbles: true  
  cancelBubble: false  
  cancelable: true  
▶ clipboardData: DataTransfer {drop  
  composed: true  
  currentTarget: null  
  defaultPrevented: false  
  eventPhase: 0  
  returnValue: true  
▶ srcElement: textarea#comentario  
▶ target: textarea#comentario  
  timeStamp: 19512.3999999976158  
  type: "copy"
```

JS: Principais Eventos HTML - conteúdo

Nome	Descrição	Gatilho
oncopy	Ocorre quando conteúdo é copiado	Copiar texto
oncut	Ocorre quando conteúdo é recortado	Recortar texto
onpaste	Ocorre quando conteúdo é colado	Colar texto

Fonte: autora, 2026

Cancelamento de comportamentos

JS: cancelamento de comportamentos

- ❖ Alguns elementos HTML possuem **comportamentos padrão** definidos pelo navegador, como o envio automático de um formulário ou a navegação ao clicar em um link.
- ❖ Em determinadas situações, pode ser necessário **impedir** essas ações utilizando o método *preventDefault()*, permitindo que o desenvolvedor execute validações ou outras **operações antes** de decidir se a ação deverá prosseguir.

```
returnValue: false
screenX: 86
screenY: 493
shiftKey: false
▶ sourceCapabilities: InputDeviceCapabilities
▶ srcElement: a#linkGoogle
tangentialPressure: 0
▶ target: a#linkGoogle
tiltX: 0
tiltY: 0
timeStamp: 18717.7000000047684
toElement: null
twist: 0
type: "click"
▶ view: Window {window: Window, self: Window,
  which: 1
  width: 1
  x: 82
  y: 413
  ▶ [[Prototype]]: PointerEvent
Navegação cancelada!
```



JavaScript

Aula 20.2: formulários, métodos get e post, protocolos http e https, obtenção de valores, validações, expressões regulares, mensagens de personalizadas e acessibilidade.

Formulários

JS: formulários

- ❖ Os formulários HTML são utilizados para **coletar informações dos usuários**, permitindo a interação entre o usuário e a aplicação.
- ❖ Eles são amplamente empregados em cadastros, autenticação, pesquisas e envio de dados para servidores.
- ❖ Um formulário é composto por **diversos campos**, como caixas de texto, listas suspensas, botões e áreas de texto.

Envie uma mensagem preenchendo o formulário abaixo

Seu Nome:

Meu Nome E Sobrenome

Seu Telefone:

999999999

Seu E-Mail:

meuemail@exemplo.com.br

Deseja receber nossas novidades?

☒ Sim ☐ Não

Sua mensagem:

Minha mensagem de exemplo!

Enviar

JS: formulários - principais atributos

- ❖ Os atributos do elemento `<form>` determinam como e para onde os dados serão enviados.

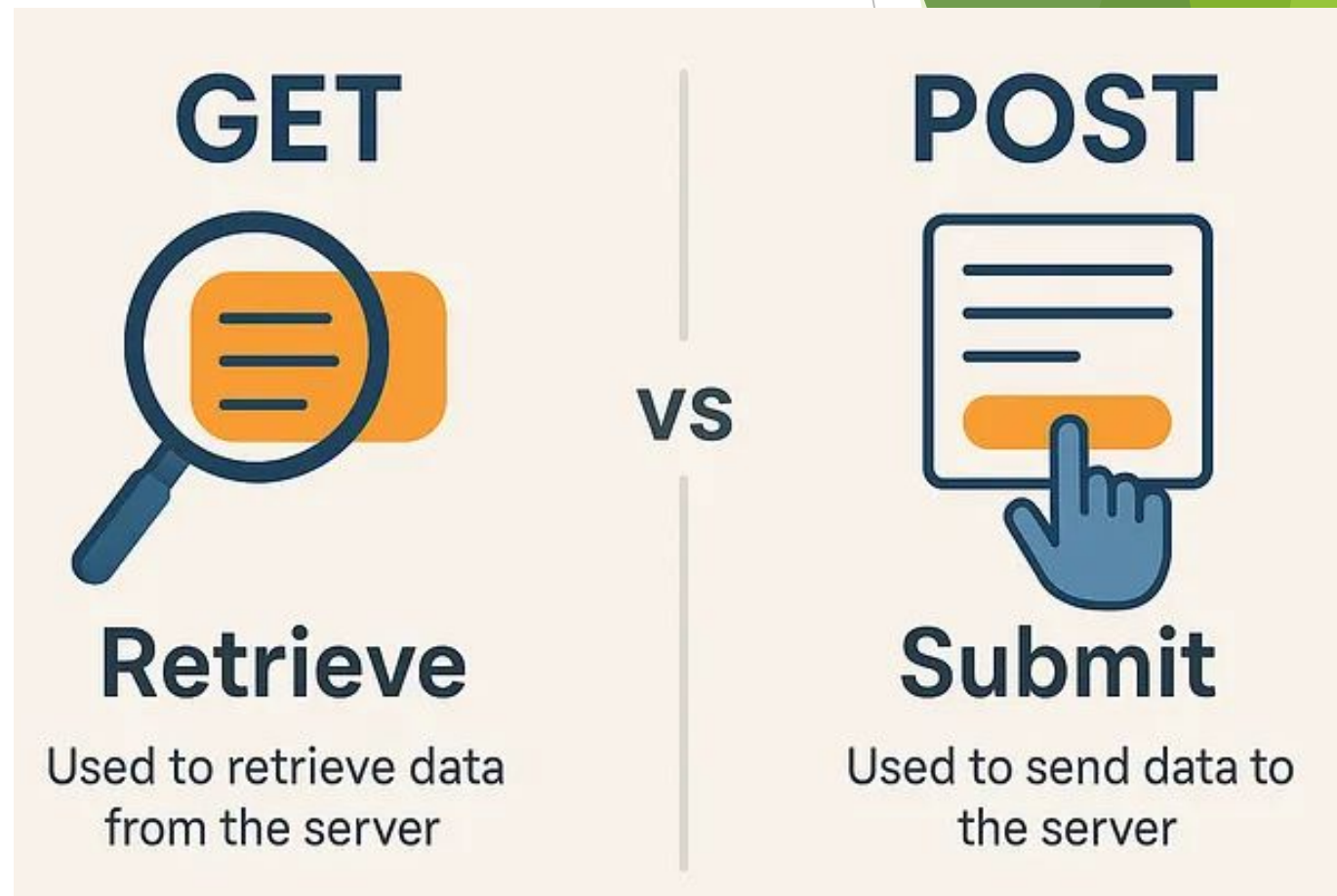
Nome	Descrição	Uso
action	Define para onde os dados serão enviados	action="/salvar"
method	Define o método HTTP (GET ou POST)	method="post"
autocomplete	Ativa ou desativa sugestões automáticas	autocomplete="off"
target	Define onde a resposta será exibida	target="_blank"
novalidate	Desativa as validações automáticas do HTML	novalidate

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The text "get x post" is centered in a green, lowercase, sans-serif font.

get x post

JS: formulários - métodos de envio

- ❖ Os formulários HTML utilizam principalmente os métodos *GET* e *POST* para enviar dados ao servidor.
- ❖ A escolha do método depende do tipo de informação enviada e da finalidade da operação.



JS: formulários - métodos de envio

- ❖ Enquanto o *GET* é mais indicado para **consultas e pesquisas**.
- ❖ O *POST* é utilizado para **enviar dados de cadastros, autenticação e outras informações que não devem ficar visíveis na URL**.



Fonte: Sharma, 2025

JS: formulários - métodos de envio

Método GET

- ❖ Envia os dados pela URL.
- ❖ Adequado para pesquisas e filtros.
- ❖ Possui limite de tamanho.
- ❖ Não deve ser utilizado para informações sensíveis.

```
<form id="formBusca" action="/buscar-produtos" method="get">  
  <label for="produto">Produto:</label>  
  <input type="text" id="produto" name="produto" required>
```

JS: formulários - métodos de envio

Método POST

- ❖ Envia os dados no corpo da requisição.
- ❖ Mais seguro que o GET.
- ❖ Não possui limitação prática de tamanho.
- ❖ Utilizado para cadastros, login e envio de arquivos.

```
<form id="formCadastro" action="/cadastro" method="post">  
  
  <!-- Nome -->  
  <label for="nome">  
    Nome completo:  
  </label>  
  
  <input type="text" id="nome" name="nome" required >
```

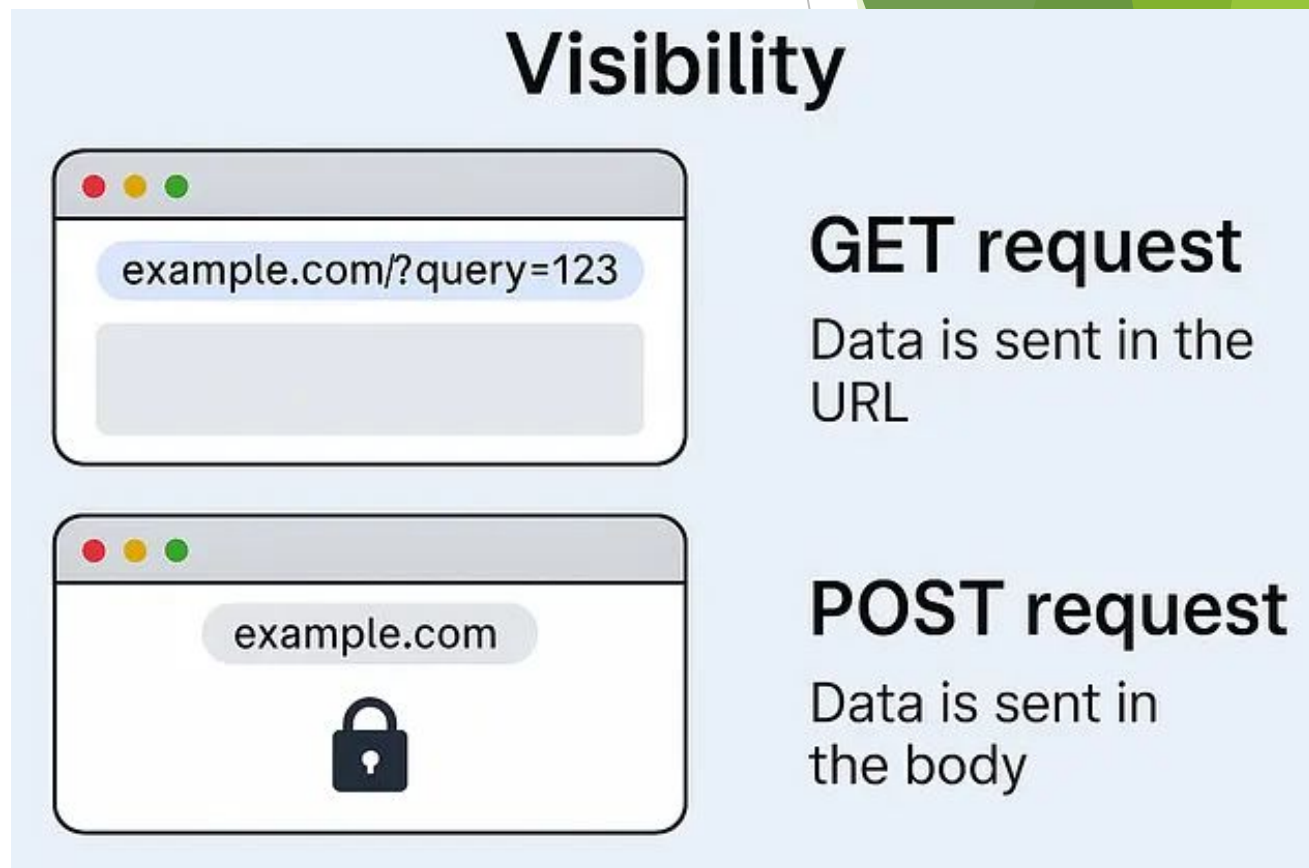
JS: formulários - métodos de envio

Nome	Descrição	URL	Visibilidade	Limitação de	Recomendação	Segurança
get	Os dados são enviados na própria URL.	site.com/busca?produto=notebook	Os dados ficam visíveis na barra de endereços	Possui limite de tamanho, pois depende do tamanho máximo da URL.	Pesquisas, filtros, paginação e compartilhamento de links.	Não recomendado para senhas e dados pessoais.
post	Os dados são enviados no corpo (body) da requisição.	site.com/login	Os dados não aparecem na barra de endereços.	Não possui limitação prática para aplicações comuns.	Cadastros, login, envio de formulários e upload de arquivos	Envia os dados no corpo (body) da requisição em vez da URL.

Fonte: autora, 2026

JS: formulários - métodos de envio

- ❖ Embora os dados enviados pelo método POST não apareçam na URL, eles não ficam automaticamente protegidos.
- ❖ Sem o uso de *HTTPS*, essas informações ainda podem ser interceptadas e visualizadas durante a comunicação entre o navegador e o servidor.



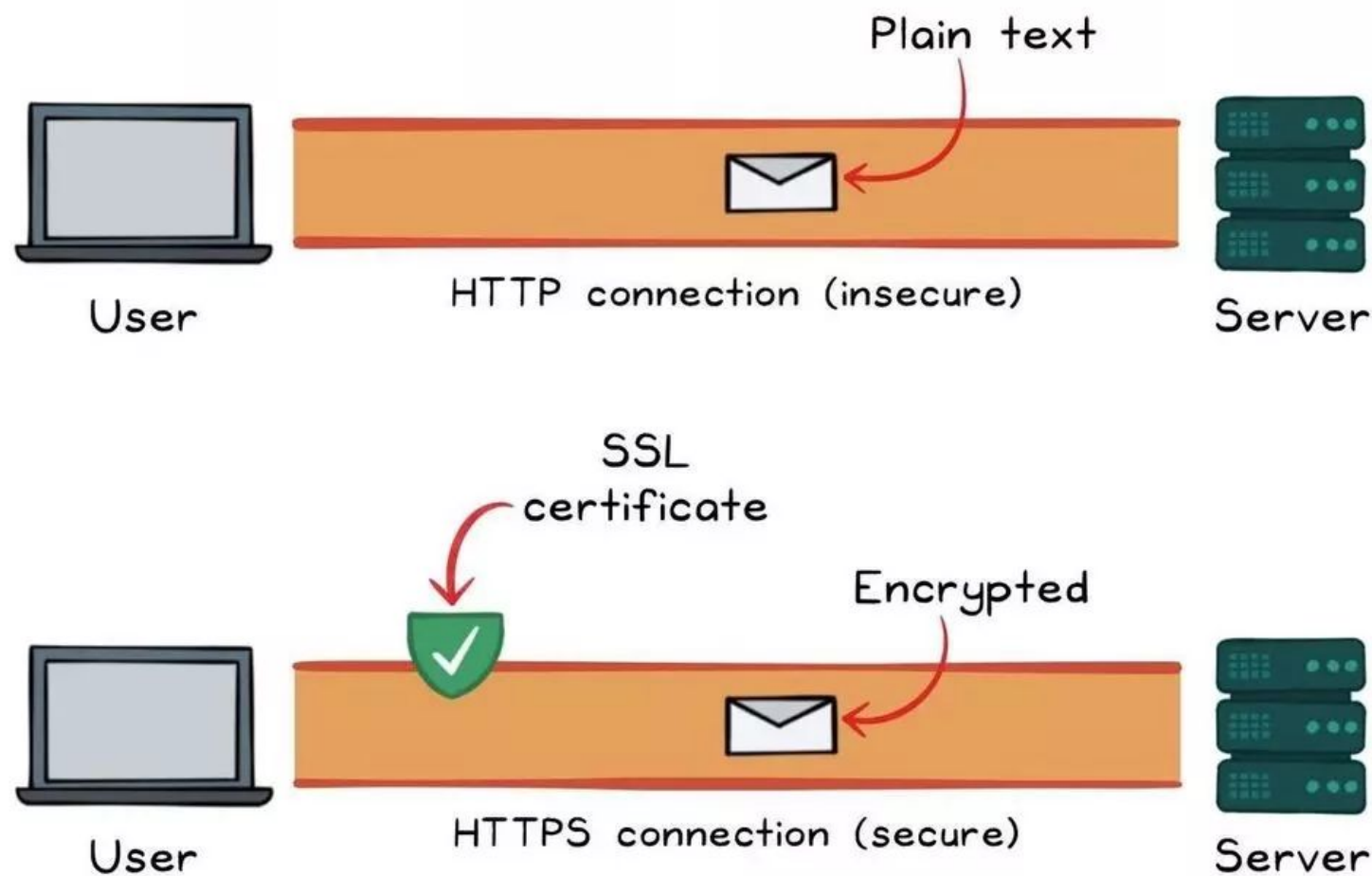
Fonte: fadatare, 2025

http x https

HTTP vs HTTPS

JS: http x https

- ❖ O método *POST* apenas altera a forma de envio dos dados, os enviado no corpo da requisição.
- ❖ A proteção efetiva depende do uso do protocolo *HTTPS*.



JS: http x https

- ❖ O **HTTPS** (*HyperText Transfer Protocol Secure*) é uma versão segura do **protocolo HTTP**.
- ❖ Ele utiliza técnicas de criptografia para **proteger a comunicação** entre o navegador e o servidor.



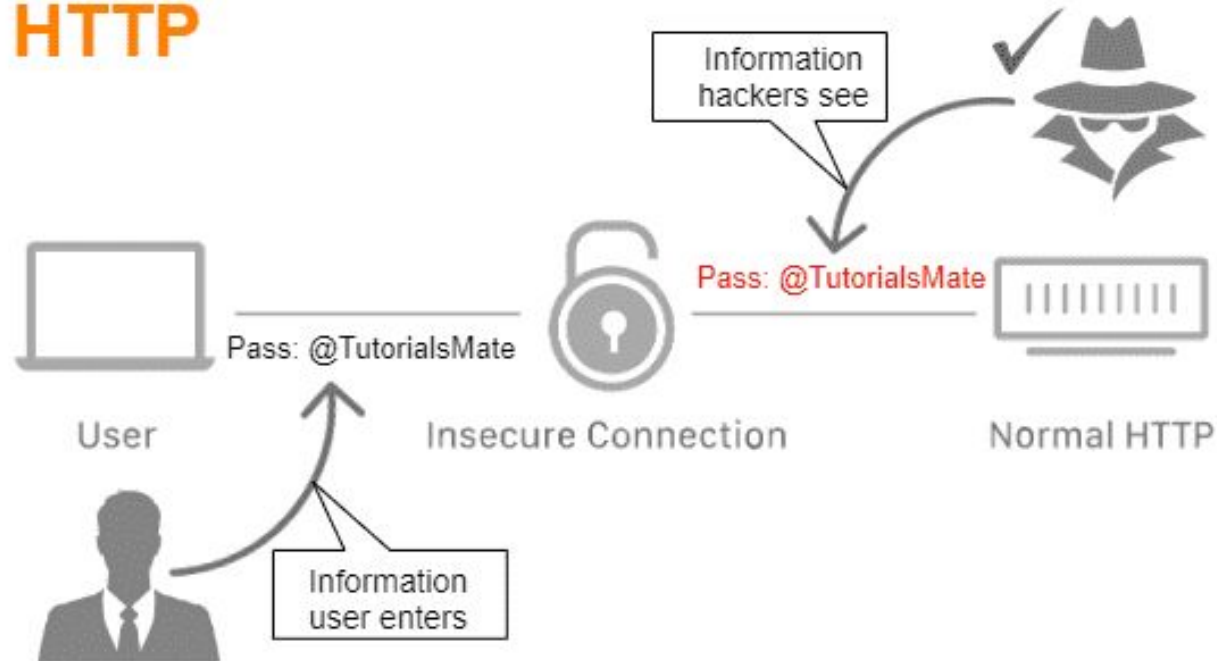
O HTTPS torna mais difícil para hackers interromperem a conexão e roubar informações pessoais, como números de cartão de crédito, endereços e senhas, por exemplo. Com isso, ajuda a proteger a privacidade de visitantes.

Fonte: Lucciani, 2023

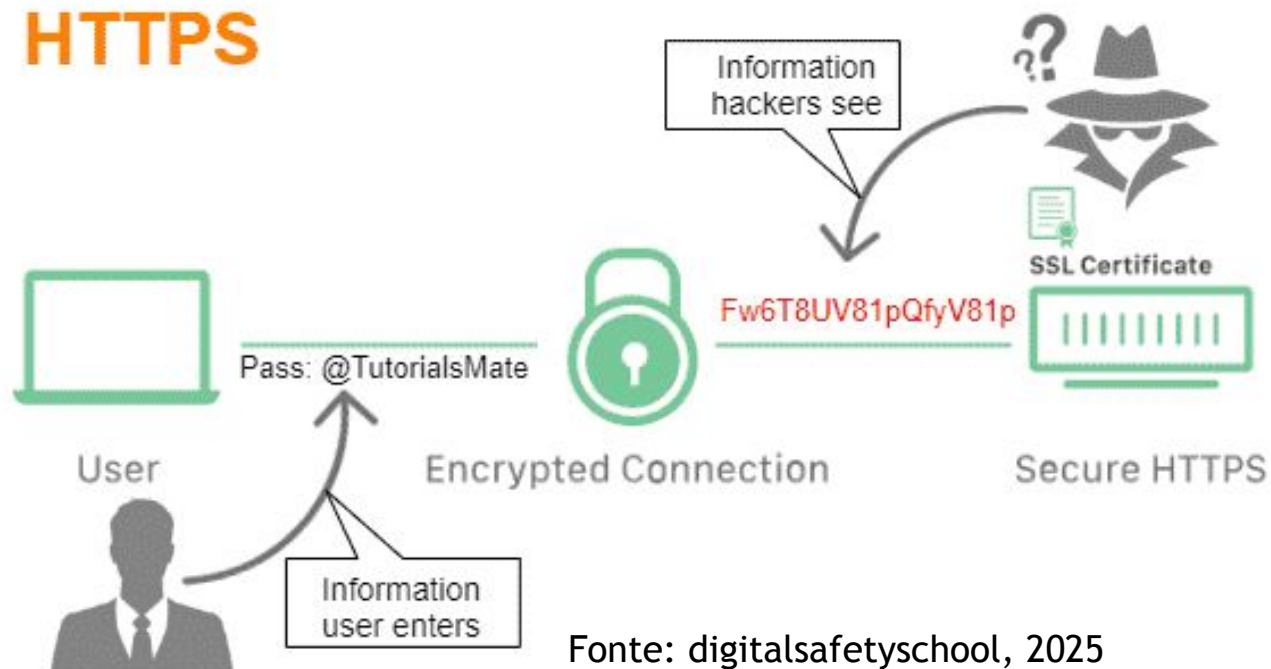
JS: http x https

- ❖ Garantindo que os dados transmitidos não possam ser facilmente lidos ou alterados por terceiros durante o tráfego na rede.
- ❖ Atualmente, o uso de *HTTPS* é essencial para qualquer aplicação web que manipule informações de usuários.

HTTP



HTTPS



Obtendo valores de campos

JS: formulários - obtendo valores de campos

- ❖ O recebimento de dados de um formulário consiste em capturar as informações digitadas pelo usuário nos campos HTML.
- ❖ Para que esses dados possam ser manipuladas e validadas antes de serem enviadas ao servidor.

```
▼ SubmitEvent {isTrusted: true, form#formBusca, ...} ⓘ  
  isTrusted: true  
  bubbles: true  
  cancelBubble: false  
  cancelable: true  
  composed: false  
  currentTarget: null  
  defaultPrevented: true  
  eventPhase: 0  
  returnValue: false  
  ▶ srcElement: form#formBusca  
  ▶ submitter: button  
  ▶ target: form#formBusca  
  timeStamp: 9365  
  type: "submit"  
  ▶ [[Prototype]]: SubmitEvent
```

JS: obtendo valores de campos de formulário

- ❖ Essa obtenção de valores é feita principalmente acessando a propriedade `.value` dos elementos do formulário.
- ❖ Ou utilizando recursos como `FormData` ou `URLSearchParams` quando os dados já foram enviados.

```
src: ""
step: ""
▶ style: CSSStyleDeclaration {accentColor: '',
tabIndex: 0
tagName: "INPUT"
textContent: ""
title: ""
translate: true
type: "text"
useMap: ""
validationMessage: ""
▶ validity: ValidityState {valueMissing: false
value: "tv"
valueAsDate: null
valueAsNumber: NaN
```

JS: obtendo valores de campos de formulário

Nome	Quando usar	Resultado	Exemplo
.value	Antes do envio	Pega 1 campo específico	<pre>const nome = document.querySelector("#nome").value;</pre>
FormData	Durante envio	Pega todos os campos do form	<pre>const dados = new FormData(form); dados.get("nome");</pre>
URLSearchParams	Após envio (GET)	Lê dados da URL	<pre>const params = new URLSearchParams(window.location.search); params.get("nome");</pre>

Fonte: autora, 2026

JS: obtendo valores de campos de formulário

- ❖ O `.value` é usado para acessar individualmente cada campo antes do envio, sendo mais simples e direto.
- ❖ O `FormData` permite coletar todos os dados do formulário de forma estruturada durante o envio, facilitando o processamento completo das informações.
- ❖ Já o `URLSearchParams` é utilizado após o envio no método `GET`, permitindo ler os dados diretamente da URL.

```
formBusca.addEventListener("submit", (event) => {  
  event.preventDefault();  
  
  // Value  
  const produtoValor = document.querySelector("#produto").value;  
  const categoriaValor = document.querySelector("#categoria").value;  
  
  // FormData  
  const dadosDoFormulario = new FormData(formBusca);  
  const produtoDadosDoFormulario = dadosDoFormulario.get("produto");  
  const categoriaDadosDoFormulario = dadosDoFormulario.get("categoria");  
  
  // URLSearchParams  
  const parametros = new URLSearchParams(window.location.search);  
  const parametrosDoFormularioProduto = parametros.get("produto");  
  const parametrosDoFormularioCategoria = parametros.get("categoria");  
});
```

Fonte: autora, 2026

Validações de formulários

JS: validações de formulários

- ❖ As validações garantem que os dados fornecidos pelos usuários estejam corretos antes do envio do formulário.
- ❖ O HTML5 oferece validações nativas, reduzindo a necessidade de código JavaScript para verificações básicas.

```
<label for="nome">
  Nome completo:
</label>

<input
  id="nome" name="nome"
  type="text"
  required
  minlength="3"
  maxlength="100"
  pattern="[A-Za-zÃ-ÿ\s]{3,}"
  aria-required="true"
  placeholder="Digite seu nome completo">
```

JS: Principais atributos de validação

Nome	Descrição	Uso
required	Torna o campo obrigatório	<input required>
minlength	Define o número mínimo de caracteres	minlength="3"
maxlength	Define o número máximo de caracteres	maxlength="50"
min	Define o valor mínimo permitido	min="18"
max	Define o valor máximo permitido	max="100"
pattern	Define uma expressão regular para validação	pattern="[A-Za-z]+"

Fonte: autora, 2026

JS: Principais atributos de validação

Nome	Descrição	Uso
type="email"	Valida automaticamente e-mails	<input type="email">
type="url"	Valida URLs	<input type="url">
type="number"	Aceita apenas números	<input type="number">
step	Define intervalos numéricos permitidos	step="0.01"

Fonte: autora, 2026

Expressões regulares

JS: expressões regulares

- ❖ O atributo *pattern* permite definir **regras de validação** para campos de formulário utilizando expressões regulares, também conhecidas como **regex**.
- ❖ Com ele, é possível definir formatos específicos para os dados inseridos, como CPF, telefone ou senhas, garantindo que as informações sigam um padrão antes de serem enviadas.

```
<label for="telefone">
  Telefone:
</label>

<input
  id="telefone" name="telefone"
  type="tel"
  required
  placeholder="479999999999"
  pattern="[0-9]{11}">
```

JS: expressões regulares

- ❖ Elas são construídas combinando **caracteres comuns** (como letras e números) com símbolos especiais, chamados de **metacaracteres**, que definem regras de *busca*, como *repetição*, *opcionalidade* ou *agrupamento*.
- ❖ Exemplo de validação de valores diversos no site [regex 101](https://regex101.com).

Regular Expression

```
:/^(\\+55)?[0-9]{11}$
```

Explanation

^ asserts position at start of a line ⓘ

1st Capturing Group (\\+55)?

? matches the previous token between **zero** and **one** times, as many times as possible, giving back as needed (greedy)

\\+ matches the character + with index **43₁₀** (**5_{3₈}** or **2B₁₆**) literally

> 55 matches the characters 55 literally

Match a single character present in the list below

[0-9]{11}

{11} matches the previous token exactly **11** times

0-9 matches a single character in the range between 0 (index **48**) and 9 (index **5**)

JS: expressões regulares

- ❖ Por possuírem uma sintaxe própria e, muitas vezes, complexa, é comum que desenvolvedores utilizem ferramentas online para auxiliar na sua construção e validação.
- ❖ Em verde, valores que dão match com a expressão regex, e em vermelho, os valores negados.

Regular Expression

Matches · 2 Steps · 58 Time · 100 μs

: / `^(\+55)?[0-9]{11}$` / gm

Test String

47999999999
+5547999999999
9999999999
5547999999999
47 9999999999
4799999-9999
01á
true
12345
teste@gmail.com

Explanation

`^` asserts position at start of a line ⓘ

✓ 1st Capturing Group `(\+55)?`

`?` matches the previous token between **zero** and **one** times, as many times as possible, giving back as needed (greedy)

`\+` matches the character `+` with index **43**₁₀ (**5**₈ or **2B**₁₆) literally

> 55 matches the characters 55 literally

✓ Match a single character present in the list below

`[0-9]{11}`

`{11}` matches the previous token exactly **11** times

`0-9` matches a single character in the range between 0 (index **48**) and 9 (index **5**)

Mensagens de erros personalizadas

JS: Mensagens de erro

- ❖ Por padrão, o HTML exibe mensagens automáticas de validação quando um campo não atende às regras definidas, como *required*, *minlength* ou *pattern*.
- ❖ Embora essas mensagens sejam úteis, elas **variam conforme o navegador** e nem sempre são claras para o usuário.

The image shows a web form with several input fields. A red rectangular box highlights the top section, which includes the 'Nome completo' field and an error message. The 'Nome completo' field contains the text 'Sa'. Below it, an error message is displayed in a white box with a red border, featuring an orange exclamation mark icon and the text 'Faça a correspondência com o formato solicitado.' Below the error message, the 'Telefone' field contains '47999999999', the 'Idade' field contains '40', the 'Senha' field is masked with dots, and the 'Linguagem favorita' dropdown menu is set to 'JavaScript'. Below these fields is a large text area for a 'Comentário' containing the word 'teste'. At the bottom of the form is a button labeled 'Enviar Cadastro'.

Nome completo: Sa

E-n  Faça a correspondência com o formato solicitado.

Telefone: 47999999999

Idade: 40

Senha:

Linguagem favorita: JavaScript ▼

teste

Comentário: Digite no máximo 200 caracteres.

Enviar Cadastro

JS: Mensagens de erro

- ❖ Para oferecer uma experiência mais amigável e consistente podemos gerar mensagens mais detalhadas e específicas para cada cenário de falha de validação de um campo.

A screenshot of a web registration form. The form fields are: 'Nome completo:' with the value 'Sa', 'E-mail:', 'Telefone:' with the value '47999999999', 'Idade:' with the value '40', 'Senha:' with masked characters, and 'Linguagem favorita:' with a dropdown menu showing 'JavaScript'. Below these is a text area for 'Comentário:' containing the word 'teste'. A red box highlights the 'Nome completo:' field and an error message: 'O nome deve possuir pelo menos 3 caracteres.' with an orange exclamation mark icon. At the bottom is a button labeled 'Enviar Cadastro'.

Nome completo: Sa

E-mail: O nome deve possuir pelo menos 3 caracteres.

Telefone: 47999999999

Idade: 40

Senha:

Linguagem favorita: JavaScript ▼

Comentário: teste

Digite no máximo 200 caracteres.

Enviar Cadastro

JS: Mensagens de erro

- ❖ Assim é possível criar mensagens personalizadas utilizando o método `setCustomValidity()` e informando o `ValidityState`.
- ❖ Permitindo informar erros de forma mais específica e adequada ao contexto da aplicação.

```
> const nome = document.querySelector("#nome");

nome.addEventListener("invalid", () => {
  if (nome.validity.valueMissing) {
    nome.setCustomValidity("O nome é obrigatório.");
  } else if (nome.validity.tooShort) {
    nome.setCustomValidity("O nome deve possuir pelo menos 3 caracteres.");
  } else if (nome.validity.patternMismatch) {
    nome.setCustomValidity("Informe apenas letras e espaços.");
  }
});
```

JS: Principais tipos de erros de validação

Estado de validação	Descrição	Exemplo
valueMissing	O campo obrigatório não foi preenchido.	Um <input required> enviado vazio.
typeMismatch	O valor informado não corresponde ao tipo esperado.	Digitar abc em um campo type="email".
patternMismatch	O valor não atende à expressão regular definida no atributo pattern.	Informar 12345 quando o padrão exige um telefone com DDD.
tooShort	A quantidade de caracteres é menor que o valor definido em minlength.	Digitar abc em um campo com minlength="8".
tooLong	A quantidade de caracteres excede o valor definido em maxlength.	Digitar 120 caracteres em um campo com maxlength="100".

JS: Principais tipos de erros de validação

Estado de validação	Descrição	Exemplo
rangeUnderflow	O valor numérico é menor que o mínimo permitido (min).	Informar 15 em um campo com min="18".
rangeOverflow	O valor numérico é maior que o máximo permitido (max).	Informar 120 em um campo com max="100".
stepMismatch	O valor não respeita o intervalo definido pelo atributo step.	Informar 2.5 em um campo com step="1".
badInput	O navegador não conseguiu converter o valor para o tipo esperado.	Digitar texto em um campo type="number".
customError	Existe uma mensagem de erro personalizada definida via setCustomValidity().	Exibir a mensagem "Senha muito fraca".
valid	Indica que o campo passou em todas as validações.	Um e-mail preenchido corretamente.

Acessibilidade

JS: Mensagens de erros personalizadas

- ❖ A **acessibilidade** em **formulários** consiste em desenvolver interfaces que possam ser utilizadas por todas as pessoas, incluindo usuários com deficiências visuais, motoras ou cognitivas.



JS: Mensagens de erros personalizadas

- ❖ Por isso é importante utilizar corretamente elementos como *label*, fornecer instruções claras, indicar **campos obrigatórios** e garantir a compatibilidade com **leitores de tela**, tornando a navegação mais inclusiva e eficiente.

```
<!-- Senha -->  
<label for="senha">  
  Senha:  
</label>  
  
<input type="password"  
  id="senha"  
  name="senha"  
  minlength="8"  
  required  
  aria-describedby="ajudaSenha">  
  
<small id="ajudaSenha">  
  A senha deve possuir pelo menos 8 caracteres.  
</small>
```

JS: Mensagens de erros personalizadas

- ❖ Todos os elementos (campos) dos formulários devem ter uma `<label>` associada.
- ❖ O uso de labels permite aumentar a área clicável do elemento incluindo também a sua legenda.



zona clicável
sem labels



zona clicável
com labels

JS: Otimizações de acessibilidade

Nome	Descrição	Exemplo
Associação entre rótulo e campo	Permite que leitores de tela identifiquem corretamente cada campo.	<code><label for="nome"> + <input id="nome"></code>
Uso de tipos específicos de entrada	Facilita a validação e melhora a experiência em dispositivos móveis.	<code>type="email", type="tel", type="number"</code>
Indicação de campos obrigatórios	Informa ao navegador e tecnologias assistivas que o campo é obrigatório.	<code>required e aria-required="true"</code>
Texto de ajuda associado	Faz com que leitores de tela leiam instruções adicionais do campo.	<code>aria-describedby="ajudaComentario"</code>
Uso de placeholders explicativos	Fornece exemplos de preenchimento (embora não substitua o label).	<code>placeholder="Digite seu e-mail"</code>

JS: Otimizações de acessibilidade

Nome	Descrição	Exemplo
Mensagens de validação personalizadas	Torna os erros mais claros e compreensíveis para os usuários.	<code>setCustomValidity()</code>
Agrupamento de campos relacionados	Organiza grupos de campos relacionados, facilitando a navegação e compreensão por leitores de tela.	<code><fieldset></code> e <code><legend></code>
Indicação de campo inválido	Informa às tecnologias assistivas que o campo contém erros de validação.	<code>aria-invalid="true"</code>
Atualização dinâmica de mensagens	Permite que leitores de tela anunciem automaticamente mensagens adicionadas dinamicamente à página.	<code>aria-live="polite"</code>

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Vídeos complementares

- ❖ Assista os vídeos abaixo para revisar o conteúdo dessa aula.



Exercício

Exercício

- ❖ Agora teste você em seu navegador os scripts vistos nessa aula.
- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos

Referência da aula

- ❖ FLANAGAN, David. JavaScript: O Guia Definitivo. 7. ed. Porto Alegre: Bookman, 2025. E-book. p.i. ISBN 9788582607008. Disponível em:
<https://integrada.minhabiblioteca.com.br/reader/books/9788582607008/>. Acesso em: 29 mai. 2026.
- ❖ MOREIRA, Sabrina B. Iniciando com JavaScript: um guia prático para entender os fundamentos da linguagem da web. 1. ed. [S.l.]: Kindle Direct Publishing, 2024. E-book Kindle. Disponível em:
<https://www.amazon.com.br/dp/B0CW9MLKTT>. Acesso em: 29 mai. 2026.

Referência da aula

- ❖ MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.119. ISBN 9788582601969. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/>. Acesso em: 11 mai. 2026
- ❖ FREECODECAMP. Web page rendering on the browser - different methods. freeCodeCamp, 2022. Disponível em: <https://www.freecodecamp.org/news/web-page-rendering-on-the-browser-different-methods/>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ EDGEMESH. How to improve your website start render time. Edgemesh Blog, [s.d.]. Disponível em: <https://edgemesh.com/blog/how-to-improve-your-website-start-render-time>. Acesso em: 30 mar. 2026.
- ❖ FADATARE, Ramesh. GET vs POST in REST API (Explained in Very Simple Words). Medium, 3 jun. 2025. Disponível em: <https://rameshfadatare.medium.com/get-vs-post-in-rest-api-explained-in-very-simple-words-a0a2c51512af>. Acesso em: 30 mar. 2026.



univille.br